

CONFIDENTIAL SECURITY AUDIT DOSSIER

Vulnerability & Risk Assessment Report

Repository Instance: test-banking-repo

BANKING RISK SCORE (BRS)	OVERALL RATING
33.0 / 100	CRITICAL POSTURE

Scan identifier: test-scan-uuid-1234

Generated Date: 2026-06-23 04:25 UTC

Compliance Basis: RBI IT Framework 2021 | PCI DSS v4.0 | SWIFT CSP

Target Environment: Production Banking Node Assessment

2. Executive Summary

ASSESSMENT STATUS: CRITICAL EXPOSURE DETECTED. Immediate patch orchestration required.

KAVACH completed a banking security audit of repository **test-banking-repo**. The scanner performed static code parsing (SAST), software composition audit (SCA), and deployment configuration assessments. A total of **3 security findings** were logged during the pipeline run.

Key Observations & Posture Analysis

- **Critical Exposures:** 1 critical-severity flaws were detected. These vulnerabilities represent immediate exploit vectors affecting banking logic, token authorization, or data integrity.
- **High Vulnerability Vectors:** 1 high-severity flaws were logged, predominantly affecting package dependencies.
- **Risk Class:** The system resides in a **Critical** risk tier with a BRS score of **33.0**.
- **Zero-Day Risk Factor:** Local dependencies and configuration audits yield a **15.0%** probability of zero-day exploitation vulnerability.

Risk Score Model Explanation

The Banking Risk Score (BRS) is a weighted calculation reflecting vulnerability severity, cvss values, and regulatory compliance mapping. BRS values above 30 denote Critical Risk (immediate remediation mandatory). BRS values between 20-30 denote High Risk (CISO review and mitigation within 7 days). BRS values below 20 denote low-to-medium posture alerts.

3. Threat Posture Overview

Banking Risk Scorecard

Vulnerability Metric	Score Value	Rating Tier	Mitigation Status
Banking Risk Score (BRS)	33.0	CRITICAL	Pending Patch
Zero-Day Risk Forecast	15.0%	LOW	Review Advisory

Threat Breakdown by Severity

Severity Level	Vulnerabilities Logged	Mitigation SLA Guide
CRITICAL	1	Immediate Patch (SLA: 24h)
HIGH	1	Mitigate within 7 days
MEDIUM	1	Mitigate within 30 days
LOW	0	Log & Track
INFO	0	Audit Reference Only

4. Regulatory Impact Analysis

Banking applications require strict adherence to regulatory IT guidelines. Findings mapped to RBI, PCI-DSS, and SWIFT CSP frameworks are highlighted below.

Compliance Standard	Violations Count	Framework Status
RBI IT Framework 2021	2	NON-COMPLIANT
PCI DSS v4.0	3	NON-COMPLIANT
SWIFT CSP	1	NON-COMPLIANT

MAPPED AUDIT DEVIATIONS

- **[CRITICAL] Hardcoded SQL Database Connection String**

MAPPED TO: RBI IT 2021: [RBI Section 5.1 - Storage of Sensitive Data](#) | PCI-DSS v4.0: [PCI DSS v4.0 Reg 2.2 - System Configuration Hardening](#) | SWIFT CSP: [SWIFT CSP Section 2.1 - Internal Data Confidentiality](#)

- **[HIGH] SQL Injection in User Login Handler**

MAPPED TO: RBI IT 2021: [RBI Section 7.2 - Secure Coding standards](#) | PCI-DSS v4.0: [PCI DSS v4.0 Reg 6.2 - Secure Software Development](#) | SWIFT CSP: [SWIFT CSP Section 4.2 - Software Security Auditing](#)

- **[MEDIUM] Outdated Cryptographic Cipher Suite (3DES)**

MAPPED TO: RBI IT 2021: [RBI Section 3.4 - Encryption Policy](#) | PCI-DSS v4.0: [PCI DSS v4.0 Reg 4.1 - Protect Cardholder Data in Transit](#)

5. Detailed Findings Section

Finding #1: Hardcoded SQL Database Connection String

Severity:	CRITICAL	CVSS Score:	9.8
Location:	config/database.py	Line:	42

Description: A hardcoded username and password was found in the database connection string. This credentials can be easily extracted by decompiling the code or inspecting files in repositories.

Business Impact: Direct compromise of the customer database. Potential theft of Cardholder Data (CHD), Personally Identifiable Information (PII), and account balances. Leads to severe RBI compliance penalties and reputational damage.

Banking Impact: Direct violation of RBI Guidelines Section RBI Section 5.1 - Storage of Sensitive Data and PCI DSS requirement PCI DSS v4.0 Req 2.2 - System Configuration Hardening and SWIFT CSP clause SWIFT CSP Section 2.1 - Internal Data Confidentiality. This critical deviation raises severe audit red flags, exposing the bank to regulatory sanctions, financial penalties by regulators, and potentially compromising user accounts.

Remediation: 1. Remove the hardcoded database credentials from code. 2. Store database secrets in a secure secret manager (e.g. AWS Secrets Manager or HashiCorp Vault). 3. Load credentials at runtime using environment variables. 4. Rotate database user passwords immediately.

Finding #2: SQL Injection in User Login Handler

Severity:	HIGH	CVSS Score:	8.8
Location:	app/auth.py	Line:	88

Description: User authentication executes queries constructed by string concatenation from user input, leading to SQL Injection.

Business Impact: Bypass authentication checks, allowing attackers to log in as admin or any customer. This leads to unauthorized banking transaction execution and fraud.

Banking Impact: Direct violation of RBI Guidelines Section RBI Section 7.2 - Secure Coding standards and PCI DSS requirement PCI DSS v4.0 Req 6.2 - Secure Software Development and SWIFT CSP clause SWIFT CSP Section 4.2 - Software Security Auditing. This critical deviation raises severe audit red flags, exposing the bank to regulatory sanctions, financial penalties by regulators, and potentially compromising user accounts.

Remediation: 1. Use parameterized queries/prepared statements for database inputs. 2. Utilize database ORM (SQLAlchemy) to automatically handle parameterization. 3. Sanitize inputs using an allowlist validation pattern.

Finding #3: Outdated Cryptographic Cipher Suite (3DES)

Severity:	MEDIUM	CVSS Score:	5.9
Location:	requirements.txt	Line:	5

Description: The application relies on 3DES cryptographic library versions which are vulnerable to collision attacks (Sweet32).

Business Impact: Potential decryption of encrypted payment payloads and customer authorization cookies. Violates modern secure transmission guidelines.

Banking Impact: Direct violation of RBI Guidelines Section RBI Section 3.4 - Encryption Policy and PCI DSS requirement PCI DSS v4.0 Req 4.1 - Protect Cardholder Data in Transit. This critical deviation raises severe audit red flags, exposing the bank to regulatory sanctions, financial penalties by regulators, and potentially

compromising user accounts.

Remediation: 1. Upgrade cipher configuration to use AES-256-GCM or ChaCha20-Poly1305. 2. Deprecate and disable 3DES cipher usage across all servers. 3. Verify SSL certificate configuration uses TLS 1.2 or TLS 1.3.

6. AI Security Analyst Commentary

Analyst Insight #1: Hardcoded SQL Database Connection String

AI Security Analyst Commentary

Threat Analysis: A hardcoded database password allows anyone with read access to the codebase to connect directly to the database. In a production system, this bypasses all network access controls and exposes all customer database records.

Attack Scenario: An adversary scans code repositories or decompiles application assets to extract hardcoded API keys/passwords. They then authenticate directly to database interfaces or APIs, bypassing internal access logs.

Risk Explanation: Direct compromise of the customer database. Potential theft of Cardholder Data (CHD), Personally Identifiable Information (PII), and account balances. Leads to severe RBI compliance penalties and reputational damage.

Recommended Action: 1. Remove the hardcoded database credentials from code. 2. Store database secrets in a secure secret manager (e.g. AWS Secrets Manager or HashiCorp Vault). 3. Load credentials at runtime using environment variables. 4. Rotate database user passwords immediately.

Analyst Insight #2: SQL Injection in User Login Handler

AI Security Analyst Commentary

Threat Analysis: String concatenation allows attackers to inject malicious SQL syntax into the database engine, bypassing login authentication checks completely.

Attack Scenario: An attacker inputs specially crafted database payloads inside UI or API fields. The application database driver executes these queries directly, allowing the attacker to bypass authentication or fetch transaction ledgers.

Risk Explanation: Bypass authentication checks, allowing attackers to log in as admin or any customer. This leads to unauthorized banking transaction execution and fraud.

Recommended Action: 1. Use parameterized queries/prepared statements for database inputs. 2. Utilize database ORM (SQLAlchemy) to automatically handle parameterization. 3. Sanitize inputs using an allowlist validation pattern.

Analyst Insight #3: Outdated Cryptographic Cipher Suite (3DES)

AI Security Analyst Commentary

Threat Analysis: 3DES utilizes 64-bit block size which is vulnerable to block collision attacks over long HTTPS connections, exposing encrypted payload bytes.

Attack Scenario: A malicious actor intercepts network packets in transit, exploiting weaker encryption protocols to perform mathematical key collisions and recover customer credentials or payload details.

Risk Explanation: Potential decryption of encrypted payment payloads and customer authorization cookies. Violates modern secure transmission guidelines.

Recommended Action: 1. Upgrade cipher configuration to use AES-256-GCM or ChaCha20-Poly1305. 2. Deprecate and disable 3DES cipher usage across all servers. 3. Verify SSL certificate configuration uses TLS 1.2 or TLS 1.3.

7. Executive Action Plan

Banking stakeholders and CISOs must execute the following remediation actions based on logged threat profiles:

PRIORITY 1: Immediate Remediation (SLA: 24h - 48h)

Remediate the 1 critical exposures logged in the codebase. Address SQL injection, token spoofing, or credential leaks immediately.

PRIORITY 2: Dependency & Configuration Patching (SLA: 7 Days)

Update the 1 outdated packages and libraries identified in the dependency audit. Focus on libraries mapped to CVEs and zero-day forecast vectors.

PRIORITY 3: Compliance & Architecture Alignment (SLA: 30 Days)

Verify complete RBI IT Framework alignment for encryption at rest and in transit. Update SBOM details and sign dependency definitions for the next regulatory audit cycle.

8. Appendix

SARIF Integration: A machine-readable SARIF v2.1.0 JSON file is generated alongside this document (test-scan-uuid-1234_findings.sarif). This can be fed into IDEs or CI/CD pipelines (GitHub Actions, GitLab CI).

SBOM Summary: Software Bill of Materials compiled in CycloneDX JSON format (test-scan-uuid-1234_sbom.json). It catalogs library trees, versions, and license profiles.

Metadata & Engine Version: KAVACH DevSecOps Core v1.0.0. Core scan time: 2026-06-23 04:25:20 UTC. Audit target: repository instance 'test-banking-repo'.

Regulatory Compliance Baseline: RBI IT Framework 2021: NON-COMPLIANT | PCI DSS v4.0: NON-COMPLIANT | SWIFT CSP: NON-COMPLIANT